

TRABAJO DE MAESTRÍA · DISEÑO Y OPTIMIZACIÓN DE BASES DE DATOS

Arquitectura Híbrida de Persistencia para Ecommify

PostgreSQL + MongoDB: decisiones de diseño, evidencia de rendimiento y análisis crítico

Autores: Oscar Clavijo · Daniel Porras · Camilo Porras

Universidad de la Sabana

Docente: Miguel Alfonso Varela

23 de junio de 2026

Contenido de la presentación

- 1 Contexto y objetivos del proyecto
- 2 Arquitectura híbrida implementada
- 3 Decisiones técnicas y su justificación
- 4 Resultados de evaluación de rendimiento
- 5 Análisis comparativo PostgreSQL vs MongoDB
- 6 Teorema CAP aplicado por módulo
- 7 Lecciones aprendidas y desafíos superados
- 8 Recomendaciones estratégicas
- 9 Conclusiones

El problema de negocio de Ecommify



El problema

Ecommify necesita soportar registro de usuarios, productos, inventario, carritos, órdenes, pagos, envíos, reseñas, búsquedas y comportamiento de usuario. No todos estos datos tienen el mismo requisito de consistencia ni el mismo patrón de acceso.



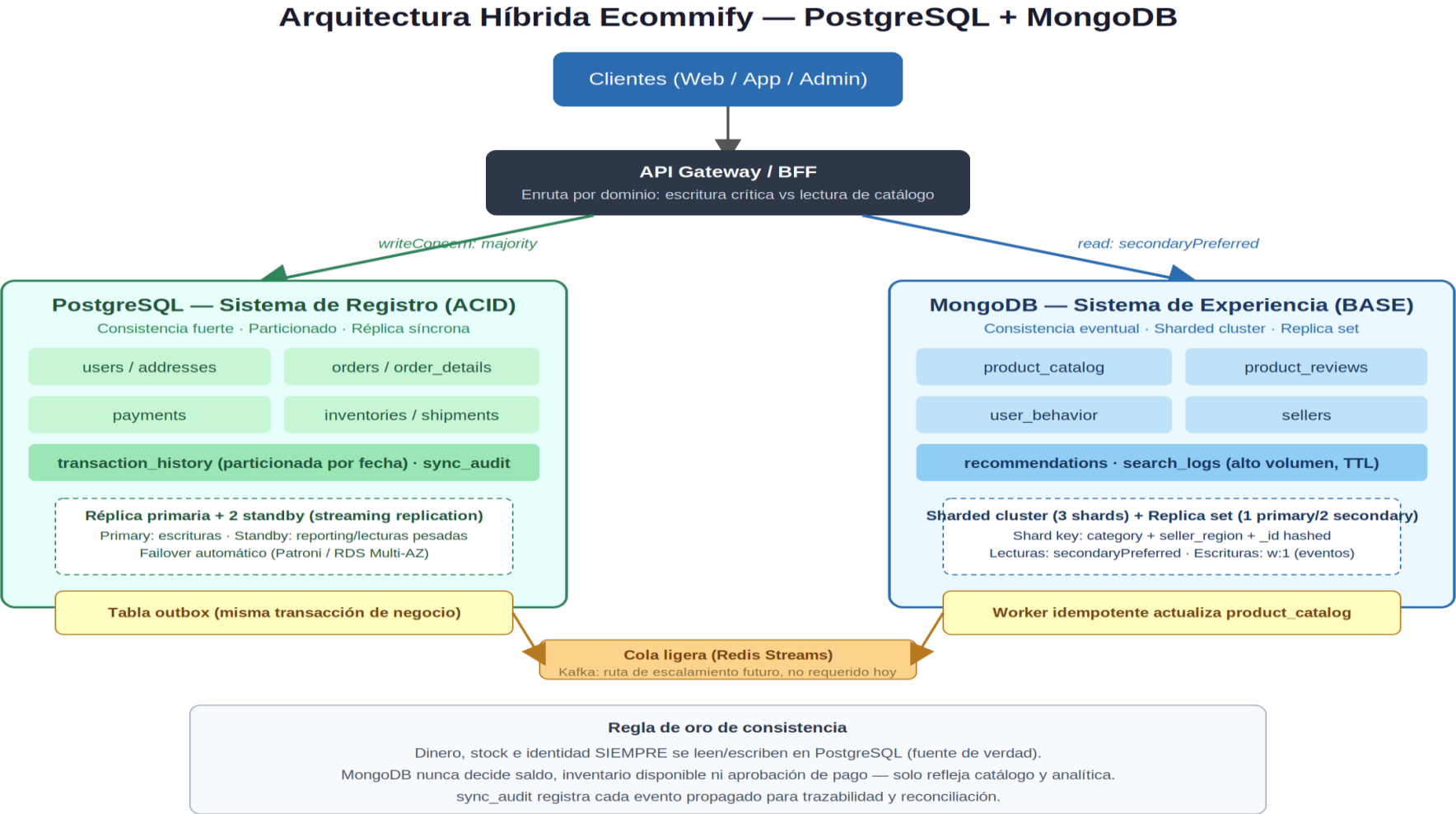
El riesgo de tratarlo igual

Pagos, órdenes e inventario no admiten inconsistencias: pueden generar impacto financiero o sobreventa. Catálogo, recomendaciones y comportamiento sí toleran consistencia eventual si la experiencia se mantiene fluida.

Objetivos del proyecto

- Diseñar una arquitectura híbrida PostgreSQL + MongoDB
- Implementar el modelo relacional crítico y el documental flexible
- Ejecutar pruebas de rendimiento, concurrencia y escalabilidad
- Comparar ambos motores con evidencia empírica
- Analizar las decisiones mediante el Teorema CAP
- Proponer recomendaciones para producción y crecimiento 10x

Arquitectura híbrida Ecommify



Sincronización PostgreSQL → MongoDB

1

La app actualiza products, inventories u orders en PostgreSQL

2

Se sincroniza hacia MongoDB mediante un proceso batch idempotente (UpdateOne + upsert)

3

product_catalog, product_reviews y user_behavior quedan actualizados

4

sync_audit registra cada corrida para trazabilidad y reconciliación

Decisión de costo: se evaluó Kafka/Debezium frente a un flujo más simple. Para el volumen real de Ecommify, un proceso batch en Python resuelve la sincronización a una fracción del costo operativo. Kafka/CDC queda documentado como ruta de escalamiento si varios sistemas necesitan consumir el mismo flujo de eventos.

Regla de oro: el checkout siempre revalida precio e inventario contra PostgreSQL antes de confirmar una compra — el catálogo de Mongo nunca decide una venta.

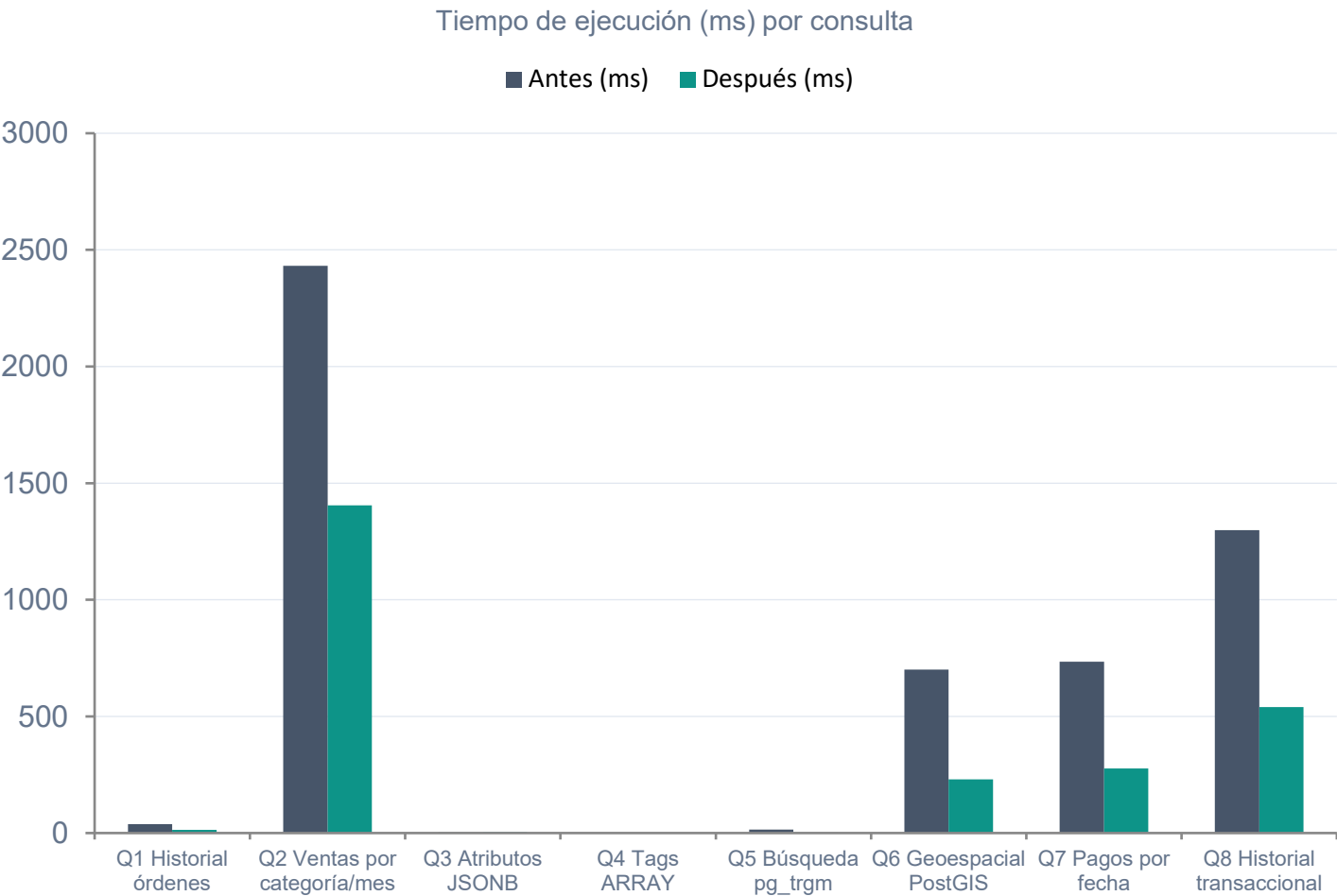
Qué fue a cada motor, y por qué

Dominio	Motor	Evidencia que lo sustenta
Usuarios, pagos, órdenes, inventario	PostgreSQL	ACID; mejoras de 58-86% con índices B-tree, BRIN, pg_trgm, GiST y partición
Catálogo, reseñas, comportamiento, recomendaciones	MongoDB	Modelo flexible; throughput competitivo bajo concurrencia moderada
Sincronización entre ambos	Outbox + batch idempotente	sync_audit permite trazabilidad y reconciliación sin Kafka

Casos límite documentados (no se ocultan, se aprenden de ellos)

Q3 (JSONB) y Q4 (ARRAY) en PostgreSQL empeoraron con índice GIN (-39.82% y -9.42%) por baja selectividad del filtro. El mismo patrón **se repite en MongoDB**: un índice no mejora el rendimiento si el filtro no reduce significativamente el conjunto de datos. La conclusión transversal es que la selectividad del filtro decide si un índice ayuda — no la tecnología.

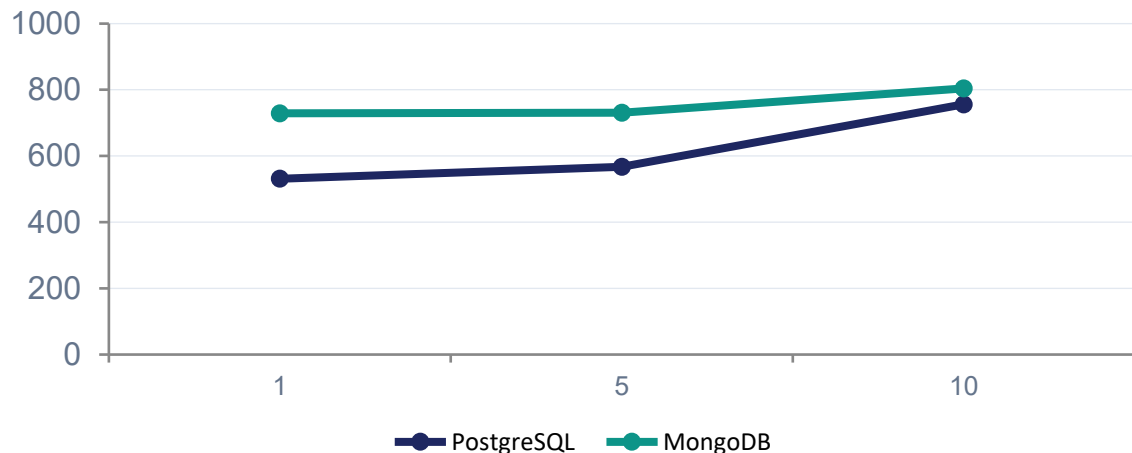
Optimización PostgreSQL: antes vs. después



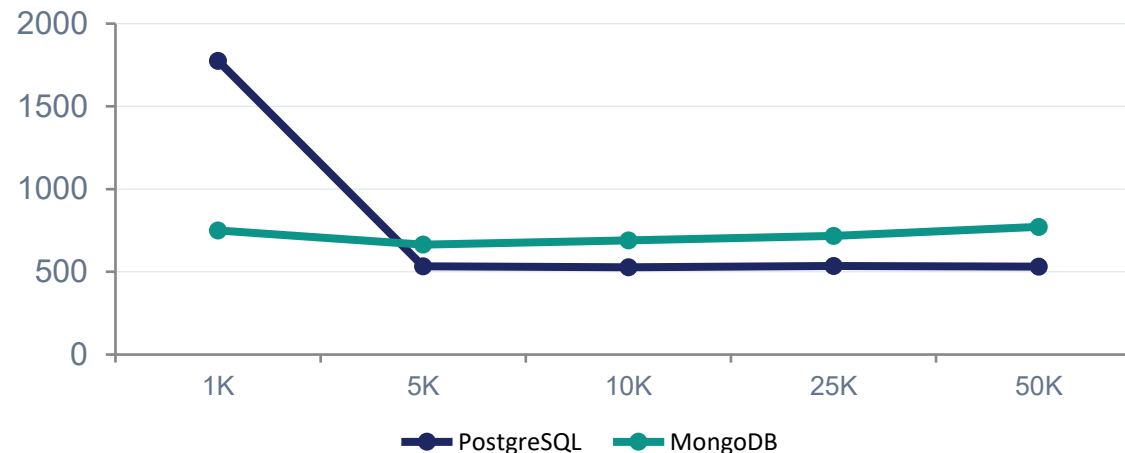
Q5 Búsqueda textual (pg_trgm)	85.70%
Q6 Geoespacial (PostGIS)	67.21%
Q1 Historial de órdenes	63.61%
Q7 Pagos por fecha	62.27%
Q8 Historial transaccional	58.44%
Q2 Ventas por categoría/mes	42.20%
Q4 Tags ARRAY	-9.42%
Q3 Atributos JSONB	-39.82%

Concurrencia y escalabilidad: PostgreSQL vs. MongoDB

Latencia promedio (ms) por usuarios concurrentes



Tiempo de respuesta (ms) por tamaño de dataset



Lectura de los resultados:

- PostgreSQL mantuvo menor latencia y mayor throughput en concurrencia controlada (1-10 usuarios); la diferencia se redujo en 10 usuarios.
- MongoDB fue más rápido con datasets pequeños (1.000 registros); PostgreSQL mostró mayor estabilidad desde 5.000 hasta 50.000 registros.
- **Punto de quiebre real: con 25 usuarios concurrentes, Supabase (free tier) cerró la conexión SSL (EDBHANDLEREXITED) — límite operativo del plan, no del diseño de índices.**

PostgreSQL vs. MongoDB en Ecommify



PostgreSQL

- Consistencia fuerte (ACID) — pagos, órdenes, inventario
- Mejoras de hasta 85.70% con índices especializados (pg_trgm, GiST, BRIN, GIN)
- Menor latencia y mayor throughput en concurrencia controlada
- Mayor estabilidad a partir de 5.000 registros en pruebas de escalabilidad
- Limitante real: el free tier administrado cierra conexiones bajo alta concurrencia (25 usuarios)



MongoDB

- Consistencia eventual aceptable — catálogo, reseñas, comportamiento, recomendaciones
- Esquema flexible: atributos variables por categoría sin migraciones
- Más rápido en el escenario inicial de 1.000 registros
- Throughput cercano a PostgreSQL en el escenario de 10 usuarios concurrentes
- Limitante real: agregaciones (lookup/pipeline) añaden 700-800 ms en ciertas pruebas

Consistencia vs. disponibilidad, módulo por módulo

Ante una partición de red, un sistema distribuido debe priorizar Consistencia (C) o Disponibilidad (A). Ecommify no aplica una sola prioridad global: la elige por módulo según el riesgo del negocio.

Módulo	Motor	Prioridad	Mecanismo
Pagos, órdenes, inventario	PostgreSQL	CP	writeConcern/transacción ACID; ante partición se rechaza antes que confirmar algo inconsistente
Usuarios y autenticación	PostgreSQL	CP	Unicidad de email; no admite duplicidad bajo partición
Catálogo de productos	MongoDB	AP	Se sirve catálogo posiblemente desactualizado antes que dar error; el precio real vive en Postgres
Comportamiento / eventos	MongoDB	AP	Se prioriza disponibilidad de ingesta; un evento perdido es reprocesable
Sincronización Postgres→Mongo	Outbox + batch	AP acotada	sync_audit detecta y reconcilia divergencias

PostgreSQL nunca pierde dinero por estar disponible. MongoDB nunca bloquea la navegación por estar desactualizado.

Lo que el proyecto enseñó en la práctica



La selectividad decide, no la tecnología

Un índice GIN/B-tree solo ayuda si el filtro reduce significativamente el conjunto de datos (Q3, Q4 y el índice de user_behavior lo confirmaron).



El rendimiento depende del ecosistema completo

No solo del motor: índices, particiones, red, región del proveedor y estrategia de conexión influyen tanto como el modelo de datos.



Los límites de un entorno administrado son reales

El punto de quiebre con 25 usuarios concurrentes en el free tier de Supabase mostró la necesidad de pooling (PgBouncer) desde etapas tempranas, no solo en producción.



La sincronización batch es viable, no ideal

El bulk_write inicial tuvo timeout; se resolvió con lotes más pequeños. Confirma que CDC/eventos es el camino natural de evolución, no un lujo.

Ruta hacia producción y escalamiento 10x

Capa	Acción recomendada
PostgreSQL	PgBouncer, réplicas de lectura, particionamiento adicional y plan productivo (no free tier)
MongoDB	Sharding activo, índices compuestos y tier dedicado M10/M20 o superior
Caché	Redis para catálogo, sesiones y resultados de búsqueda frecuentes
Búsqueda	OpenSearch / Elasticsearch para búsqueda textual avanzada y facetas
Sincronización	Evolucionar de batch a Kafka / Debezium / CDC cuando varios sistemas consuman el mismo evento
Observabilidad	Prometheus, Grafana y Atlas Monitoring con alertas sobre conexiones y replication lag

La recomendación principal es mantener la arquitectura híbrida — no migrar todo a un solo motor.

Lo que demuestra el proyecto



La arquitectura híbrida PostgreSQL + MongoDB fue la decisión más equilibrada para Ecommify: una solución 100% relacional sería rígida para datos flexibles; una 100% NoSQL elevaría el riesgo en pagos, órdenes e inventario.



Cada asignación de dominio a un motor está sustentada en evidencia empírica medida (EXPLAIN ANALYZE, pruebas de concurrencia y escalabilidad), no en preferencia tecnológica.



El Teorema CAP no se aplicó como una etiqueta global, sino módulo por módulo, según el riesgo real de negocio de cada dato.



La arquitectura híbrida solo funciona si se complementa con sincronización confiable, monitoreo de conexiones y un plan de evolución hacia eventos (CDC) a medida que el negocio crece.

Gracias

Arquitectura Híbrida de Persistencia para Ecommify

Oscar Clavijo · Daniel Porras · Camilo Porras — Universidad de la Sabana

Diseño y Optimización de Bases de Datos — Docente Miguel Alfonso Varela